

Energy Measurements, Metrics, and Models in Communication Networks

Tobias Hoßfeld, Frank Loh
info3.org

Tutorial Agenda

► Energy consumption measurements

- How can we measure energy consumption in theory?
- External and internal measurements
- Challenges in high precision energy measurements
- **Example:** impact of imprecise energy measurements

► Energy efficiency metrics and modeling

- Energy efficiency metric overview
- Energy intensity
- Consumption and output related energy efficiency
- Energy modeling

MEASURING ENERGY CONSUMPTION IN COMMUNICATION NETWORKS

Measuring Energy Consumption in Communication Networks

► Why?

- To save energy and cost, to emit less carbon ...
- ... many more reasons

► So why not just buying the most efficient device using cheap renewable energies?

► So why should we really measure energy in communication networks?

- **It might be costly!** You need devices and additional effort
- **It might be time consuming!** You need expertise and time for deployment
- **Most information is already there!**
 - ‘The expected battery lifetime of LoRa sensors is 10 years’ (in many research papers)
 - The network device consumes 800W (or however much) under load

► Additional questions

- How much energy is required in particular operational states?
- How much energy is required during 80% load? Or 60%?

► We measure energy: to comprehensively understand our systems and perform optimization!

ENERGY MEASUREMENTS IN THEORY

Example Energy Measurements in Communication Networks

► Setup in practice rather simple...

- Device under test
- Power meter
- For detailed measurements: some form of traffic generation

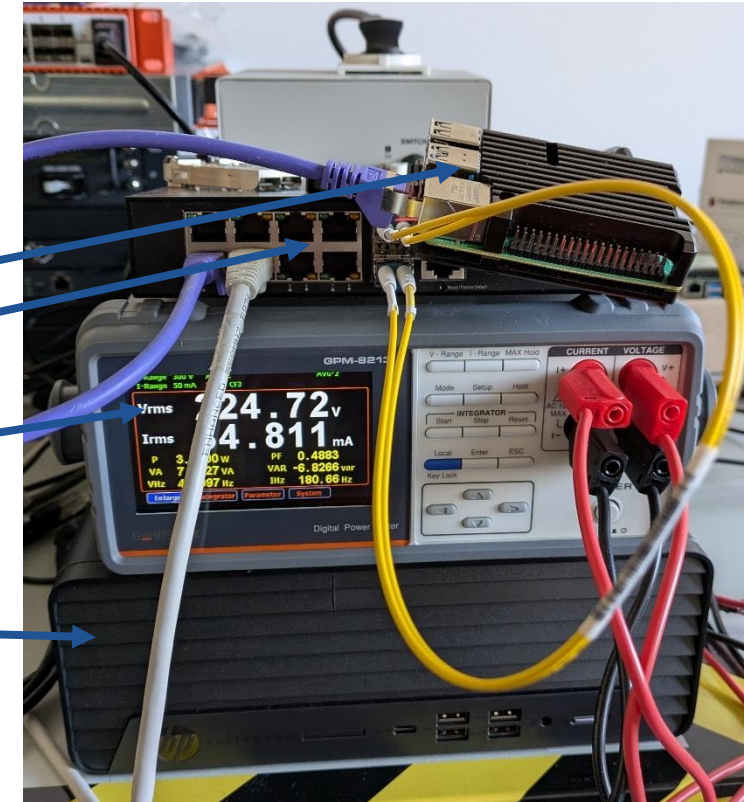
► Example setup

- Traffic destination: Raspberry Pi
- DUT: Edge-Core ECS2100-10T
- Power meter: GW-Instek GPM8213
- Traffic source: PC

► Possible energy measurements

- Measurements in idle and under different load conditions
- Measurements with different ports in use
- Measurement for different traffic characteristics

► ... if **understood** and **calibrated** correctly using the correct measurement device for the use case!



Energy Measurements: Fundamental Principles

- ▶ Measuring energy consumption requires capturing the power P used over time
- ▶ Energy is the integral of **power over time**
 - Measure **voltage** V and **current** I
 - Calculate instantaneous power
 - Sum it up over the measurement interval
- ▶ Power P at any instant is $P(t) = V(t) \cdot I(t)$
- ▶ Energy E over a time interval is

$$E = \int_{t_0}^{t_1} P(t) dt = \int_{t_0}^{t_1} V(t) \cdot I(t) dt$$

Direct and Alternating Current

► Alternating Current (AC)

- Electric charge flow **reverses direction periodically**
- **Voltage & Current:** Alternates between positive and negative values, usually in a **sine wave**
- **Source examples:** Power grids, wall sockets (50 Hz in Europe, 60 Hz in the US)
- **Typical usage:** Household appliances, industrial motors, long-distance power transmission

Alternating Current in Detail: Resistive Loads

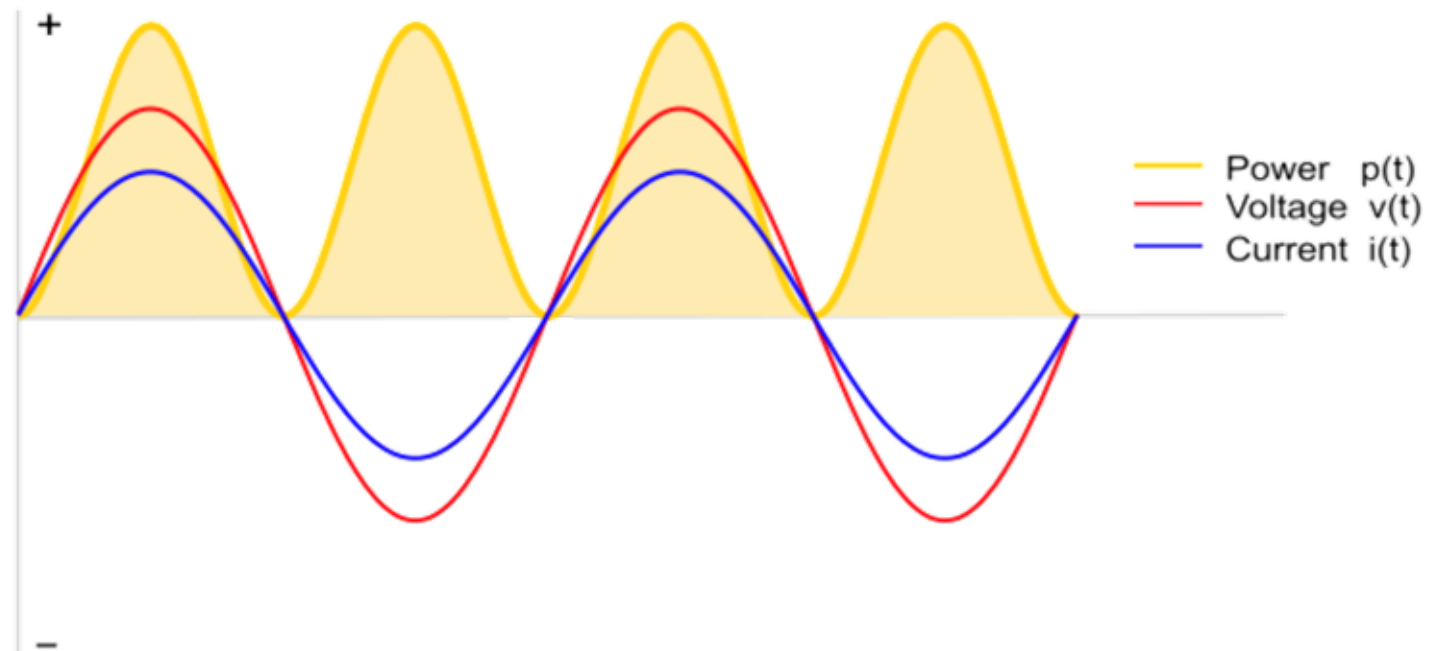
- ▶ Not all device 'interact' the same way with the power supply

- ▶ **Resistive loads**

- Some devices use all energy given to them (e.g., to light your building or make your toast)
- Directly converted to heat or light
- They are **resistive loads**, i.e., their current draw is equal to the voltage divided by their resistance (Ohm's law)
- Their power is always positive

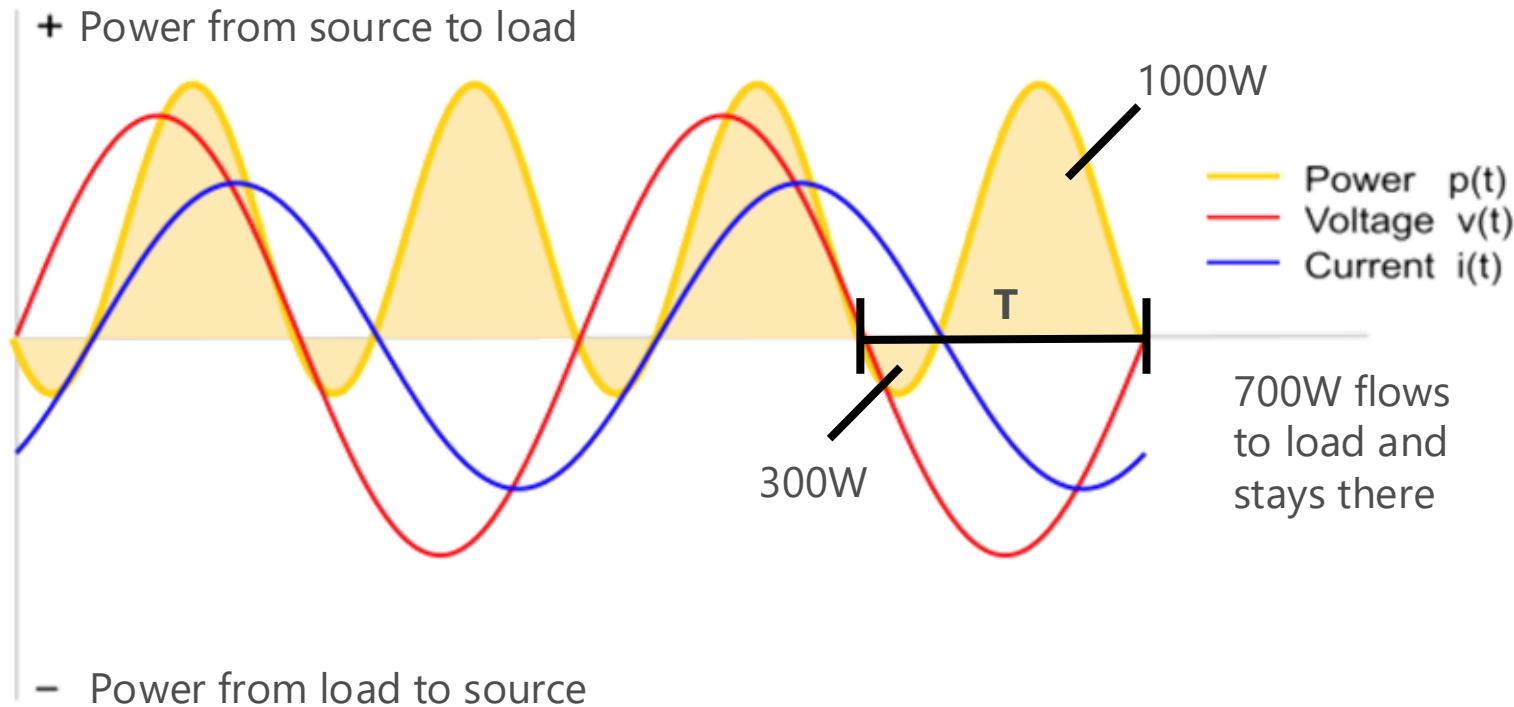
- ▶ **Example devices**

- Water heaters
- Electric cookers
- Toasters
- Light bulbs



Alternating Current in Detail: Besides Resistive Loads

- ▶ Devices like fridges, washing machines, many switches, and routers are less straight forward
- ▶ **(Partially) reactive loads** take in a certain amount of energy
- ▶ Then, theoretically, release some back to the energy network



practical example

Foam: **reactive power**
useless

Beer: **real power**
useful, does the work

The glass: **apparent power**



$$\text{Power factor} = \frac{\text{real power}}{\text{apparent power}} \quad \text{or} \quad \text{Power factor} = \frac{\text{how much beer}}{\text{per glass we buy}}$$

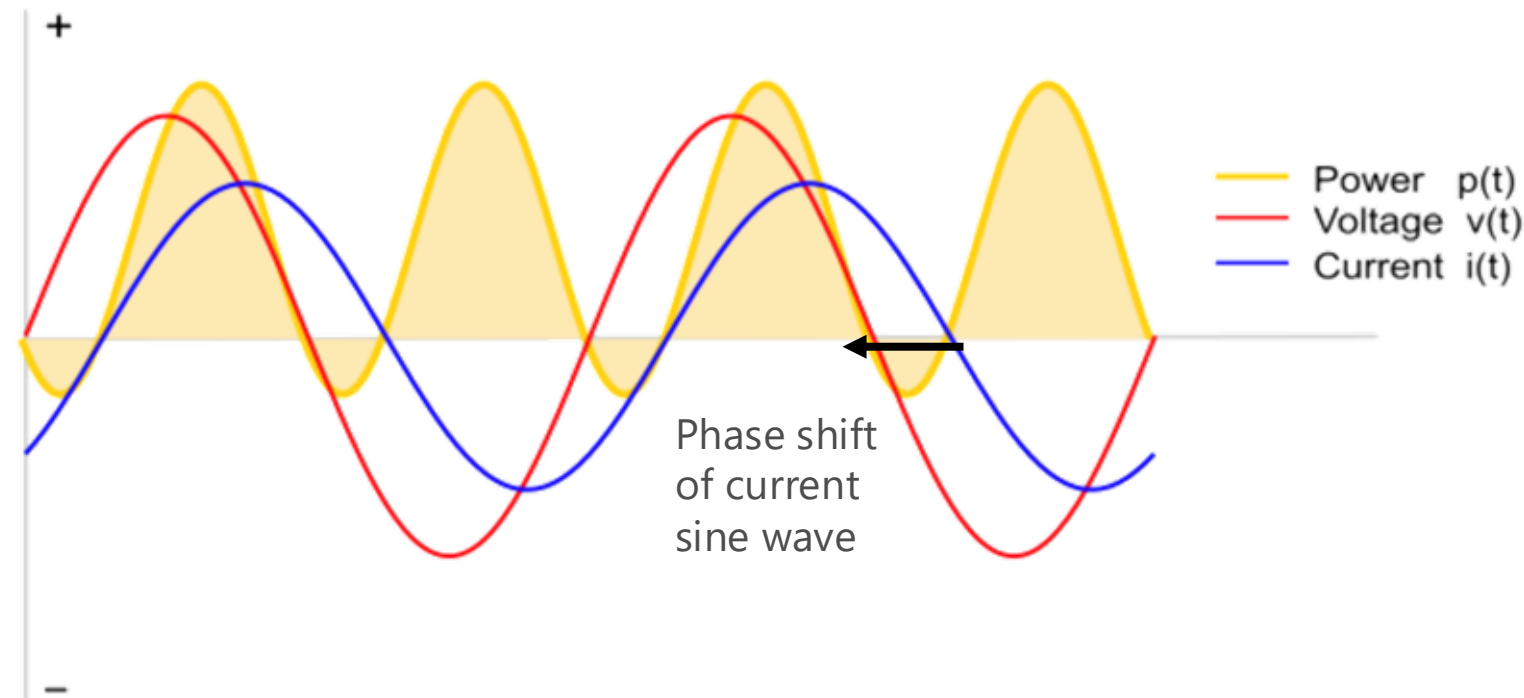
- ▶ What is **the reason** for this behavior?

Alternating Current in Detail: Inductive Loads

► Inductive loads

- Many devices such as motors and fans, use coils to produce magnetic fields
- When energized, created magnetic fields delays current relative to the voltage
- This leads to a phase shift between current and voltage sine waves
- **In addition:** components or devices that store energy are required
- Especially electric motors required high 'inrush' current on start-up
- Often much higher than operating current

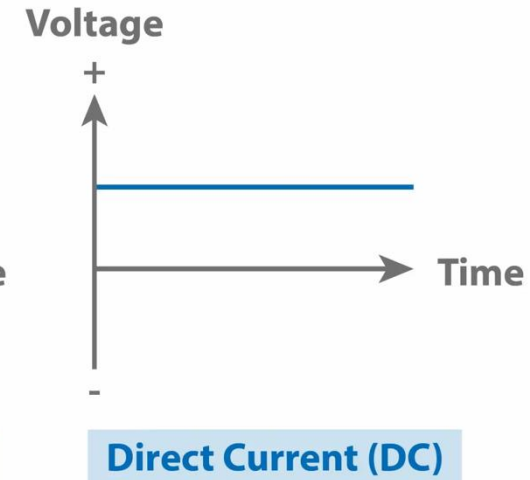
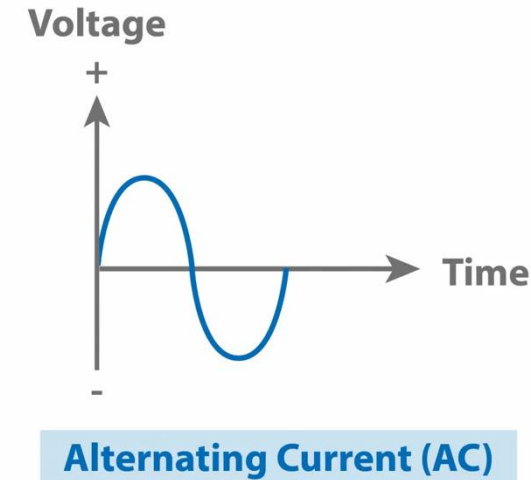
- So most real devices in networks, e.g., switches, use active power and reactive power (stored and released)



Direct and Alternating Current

► Alternating Current (AC)

- Electric charge flow **reverses direction periodically**
- **Voltage & Current:** Alternates between positive and negative values, usually in a **sine wave**
- **Source examples:** Power grids, wall sockets (50 Hz in Europe, 60 Hz in the US)
- **Typical usage:** Household appliances, industrial motors, long-distance power transmission



► Direct Current (DC)

- Electric charge flows **only in one direction**
- **Voltage & Current:** Voltage is (ideally) constant, and current has a fixed polarity
- **Source examples:** Batteries, solar cells, fuel cells, USB chargers
- **Typical usage:** Electronics (IoT devices, laptops, smartphones), LED lighting, automotive systems
- Typically, **easier to measure!**

Energy Measurement using Direct Current

- ▶ To perform energy measurements, you need both voltage and current measurements

1. Voltage measurement

- Usually straightforward: measure voltage **in parallel** to the component or source
- General voltage source often known in a setup, e.g., using a 3V battery
- **Careful:** capacity is dropping with battery age
- Frequent calibration measurements required, or battery age must be considered

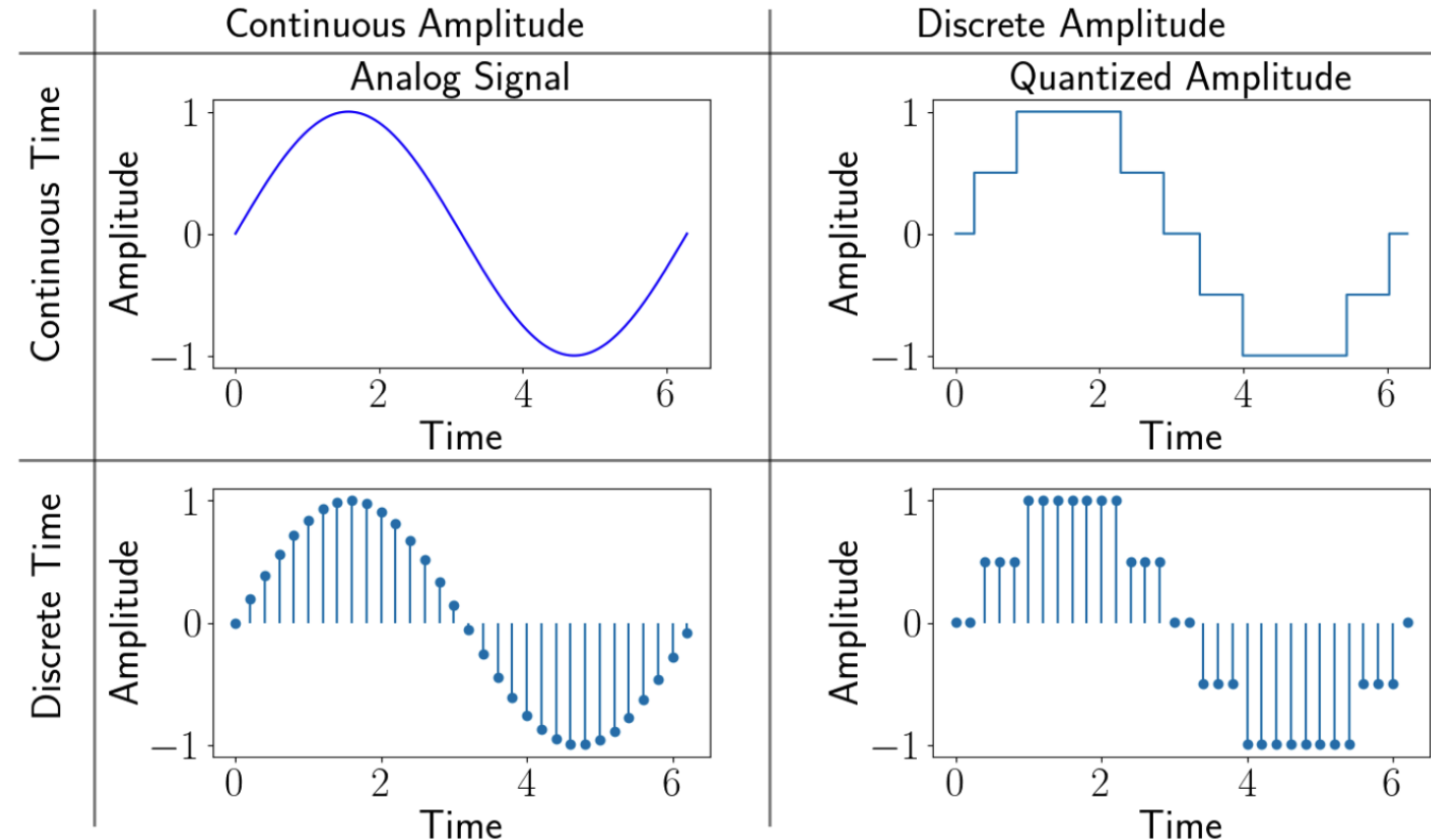
2. Current measurement

- Commonly done using a **shunt resistor** in **series** with the device
- The resistor creates a small but known voltage drop proportional to current ($V_{shunt} = I \cdot R_{shunt}$)
- A differential amplifier or sensor (e.g., INA226, INA219) converts this to a readable signal

- ▶ To this point: both **voltage** and **current** are **continuous** in **time** and **amplitude**

From Continuous to Discrete Time and Amplitude Values

- ▶ In practical applications, physical signals are continuous in time and amplitude
- ▶ **Sampling**
 - To discretize time
 - Frequent enough to capture dynamic changes
 - **Nyquist criterion:** at least two times the highest frequency in the signal
- ▶ **Quantization**
 - To discretize amplitude
 - Finite set of different values



- ▶ In practice, thus, energy is a discrete sum after sampling and quantization

$$E \approx \sum_{k=1}^N V[k] \cdot I[k] \cdot \Delta t + \epsilon \text{ with } \Delta t \text{ as the } \mathbf{\text{sampling period}}$$

(LOW COST) EXTERNAL ENERGY MEASUREMENTS IN THE LAB

How to Set up Low-Cost External Energy Measurements

- ▶ **Energy measurement setup in practice rather simple...**
 - Device under test
 - Power meter
 - For detailed measurements: some form of traffic generation
- ▶ ... if **understood** and **calibrated** correctly using the correct measurement device for the use case!
- ▶ What if we do not have any expensive, high accuracy, high power energy measurement setup?
- ▶ **Cheaper solution:** Tinkerforge energy measurement setup

- ▶ **Tinkerforge setup**

- Master Brick connected to measurement PC
- Voltage/Current Bricklet 2.0 for energy measurements
- Voltage and current measured in an alternating manner
- Measurement triggered via API calls, duration is configurable

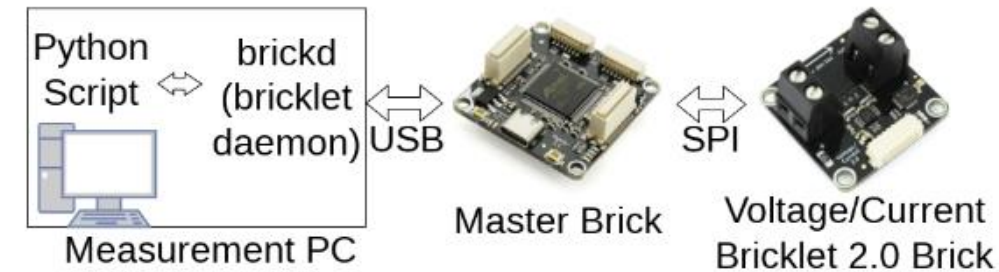


Figure from [1]

Tinkerforge Energy Measurements: Some Results

- ▶ Energy measurements for different Radio Access Network technologies using the Tinkerforge setup

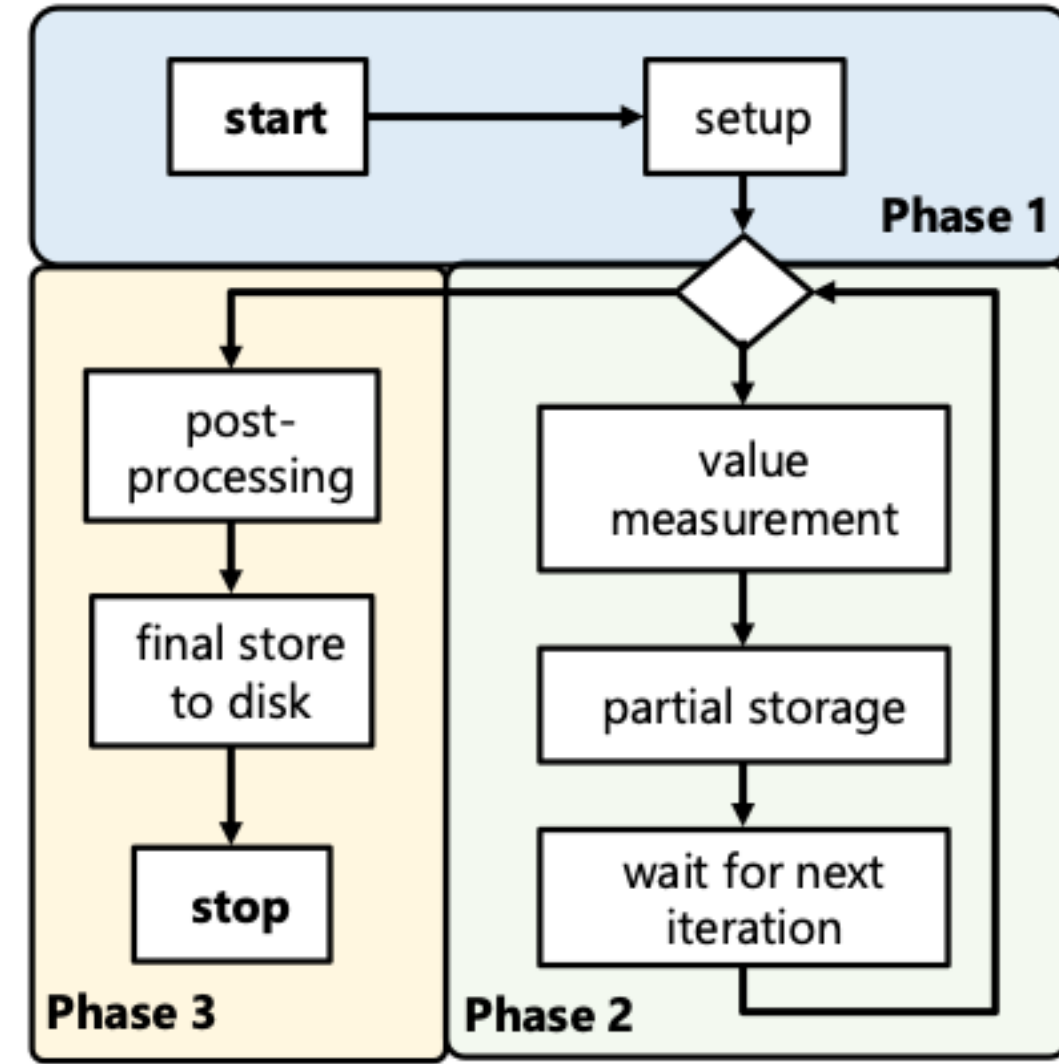
Payload	2G			4G			NB-IoT			LTE-M		
10 B	1.45 s	0.53 W	0.75 J	0.42 s	0.86 W	0.35 J	2.81 s	0.47 W	1.31 J	0.77 s	0.78 W	0.60 J
100 B	1.37 s	0.56 W	0.74 J	0.35 s	0.89 W	0.31 J	2.80 s	0.47 W	1.31 J	0.80 s	0.79 W	0.64 J
1 kB	1.80 s	0.53 W	0.92 J	0.37 s	0.81 W	0.30 J	4.36 s	0.47 W	2.01 J	1.25 s	0.80 W	0.90 J
10 kB	3.11 s	0.73 W	2.00 J	0.39 s	0.86 W	0.33 J	5.99 s	0.51 W	3.01 J	1.91 s	0.99 W	1.89 J
100 kB	8.09 s	1.25 W	9.63 J	0.73 s	1.06 W	0.77 J	23.93 s	0.66 W	15.64 J	9.19 s	1.16 W	10.67 J
1 MB	62.76 s	1.58 W	91.67 J	2.37 s	1.56 W	3.67 J	209.94 s	0.71 W	148.53 J	74.33 s	1.21 W	89.55 J

Results from [2]

- ▶ Various insights into the **different technologies** and **energy consumption** for **transmissions**
 - Transmission with 4G consumes least energy
 - LTE-M as Low Power Wide Area technology requires less energy than NB-IoT for transmission
 - Longer transmissions require, as expected, more energy
- ▶ **In this example:** comparably long measurements (350ms+)
 - How accurate are the measurements with the Tinkerforge setup for very **short power spikes**?
 - What and where are the **limitations**? How to identify them?

Tinkerforge Energy Measurement Setup: Further Details

- ▶ Energy measurement procedure in three phases
 - **Phase 1:** setup phase
 - **Phase 2:** actual measurement phase
 - **Phase 3:** processing and clean up phase
- ▶ Phase 2 mainly relevant for current and voltage measurements
- ▶ **In each loop**
 - Current and voltage is measured x times
 - For a duration of t each
 - Each loop is designed to match a target time T
 - Results are averaged and written to registers
 - After the target time has passed, the next measurement cycle can start



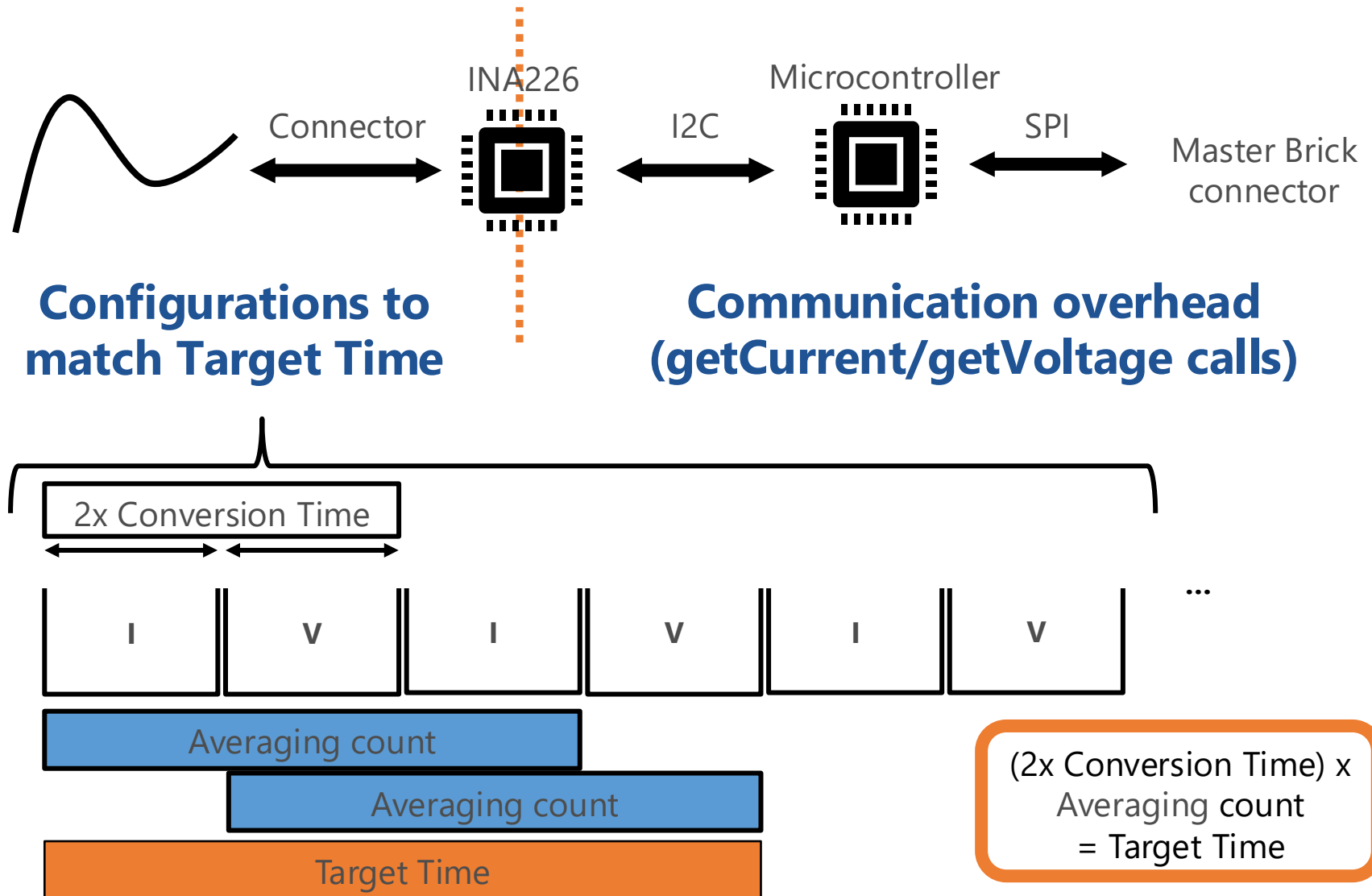
Tinkerforge Voltage/Current Bricklet 2.0 in more Detail

- ▶ Alternating current/voltage measurements started with 'getCurrent()/getVoltage()' call

- ▶ **Two error sources**

- General setup leading to latencies, communication overhead and timing errors
- Configuration options to match target time

- ▶ **Example:** when I measure current, I can not measure voltage with the setup at the same time



Tinkerforge Setup: Possible Configuration

- ▶ **Conversion time** possible between $140\mu\text{s}$ and 8.244ms
- ▶ **Averaging** possible between 1 and 1024

Default config

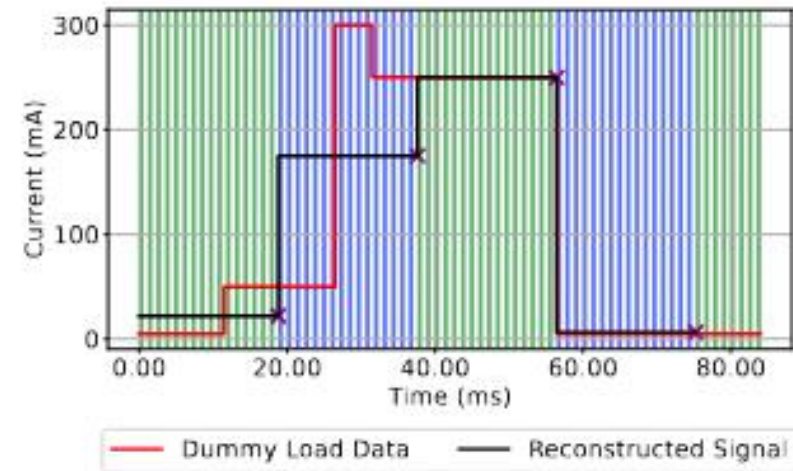


- ▶ Tested **different conversion times** and **averaging values** leading to **different target times** for energy measurements
- ▶ **For shorter target times**
 - Duration `getCurrent()` and `getVoltage()` calls longer than target time
 - **Thus:** timing errors from overhead

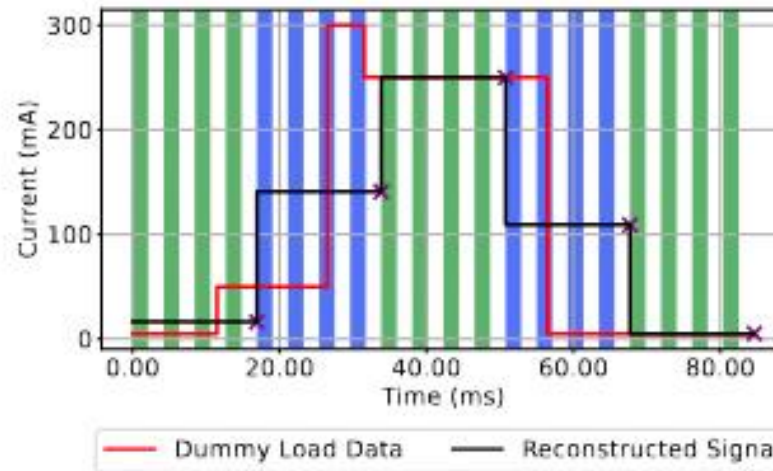
Target Time	Conversion Time	Averaging
1.632ms	204us	4
2.2ms	1.1ms	1
2.656ms	332us	4
4.323ms	2.116ms	1
4.48ms	140us	16
4.704ms	588us	4
8.312ms	4.156ms	1
8.8ms	1.1ms	4
10.336ms	332us	16
16.488ms	8.244ms	1
16.928ms	2.116ms	4
18.816ms	588us	16

Impact of Conversion Time and Averaging Count

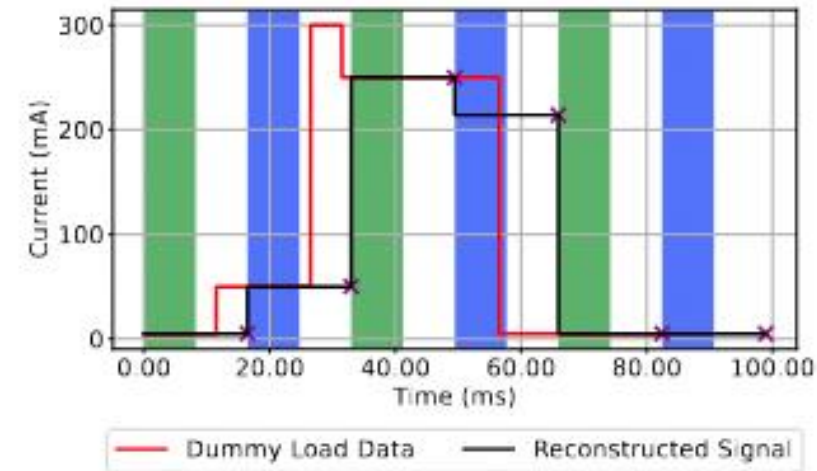
Avg Count 16, Conversion Time: 0.588ms



Avg Count 4, Conversion Time: 2.116ms



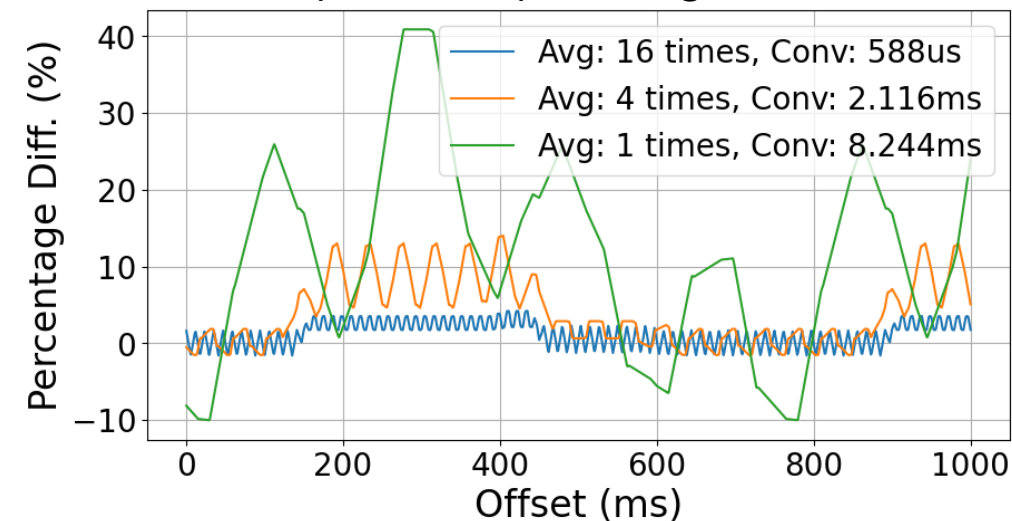
Avg Count 1, Conversion Time: 8.244ms



Results from [1]

- ▶ Real, dummy load in red, reconstructed, measured signal in black
- ▶ Large discrepancy between real and measured signal
- ▶ Different errors made, dependent on conversion time and averaging count
- Requirement of correct setup with correct settings!

Comparison of percentage difference



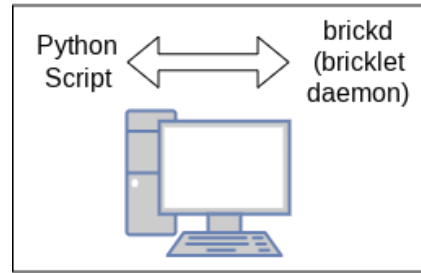
Improve Settings: Shorter Target Time?

- ▶ Shorter conversion time and smaller averaging leads to shorter target time
- ▶ **In theory**, better results because of less averaging effect
- ▶ **In the Tinkerforge setup**
 - API calls `getVoltage()` and `getCurrent()` to start measurements
 - Such calls induce additional delay
 - Delay must be shorter than target time
- ▶ **For short target times**
 - API call duration longer than total target time
 - Leads to timing accuracy errors

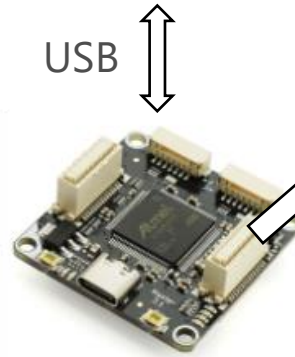
Target time	Timing accuracy error	Jitter	Proportion of API call
1.632ms	22.707%	3.808%	121.504%
2.2ms	2.699%	19.756%	87.942%
2.656ms	0.402%	18.078%	73.927%
4.232ms	0.312%	10.528%	47.925%
4.48ms	0.291%	11.451%	45.203%
4.704ms	0.282%	9.871%	43.070%
8.312ms	0.212%	5.906%	25.990%
8.8ms	0.204%	4.811%	24.777%
10.336ms	0.183%	4.821%	21.685%
16.488ms	0.112%	2.991%	14.955%
16.928ms	0.114%	2.054%	14.641%
18.816ms	0.105%	1.997%	13.315%

Comparison of Tinkerforge Setup to High Precision Energy Measurements

- ▶ Comparison of Tinkerforge setup to high precision current/voltage measurements
- ▶ **Current reference:**
Nordic Power Profiler Kit 2
- ▶ **Voltage reference:**
Saleae Logic Pro 16
- ▶ Separate devices for current and voltage reference measures

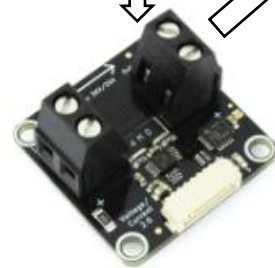


USB

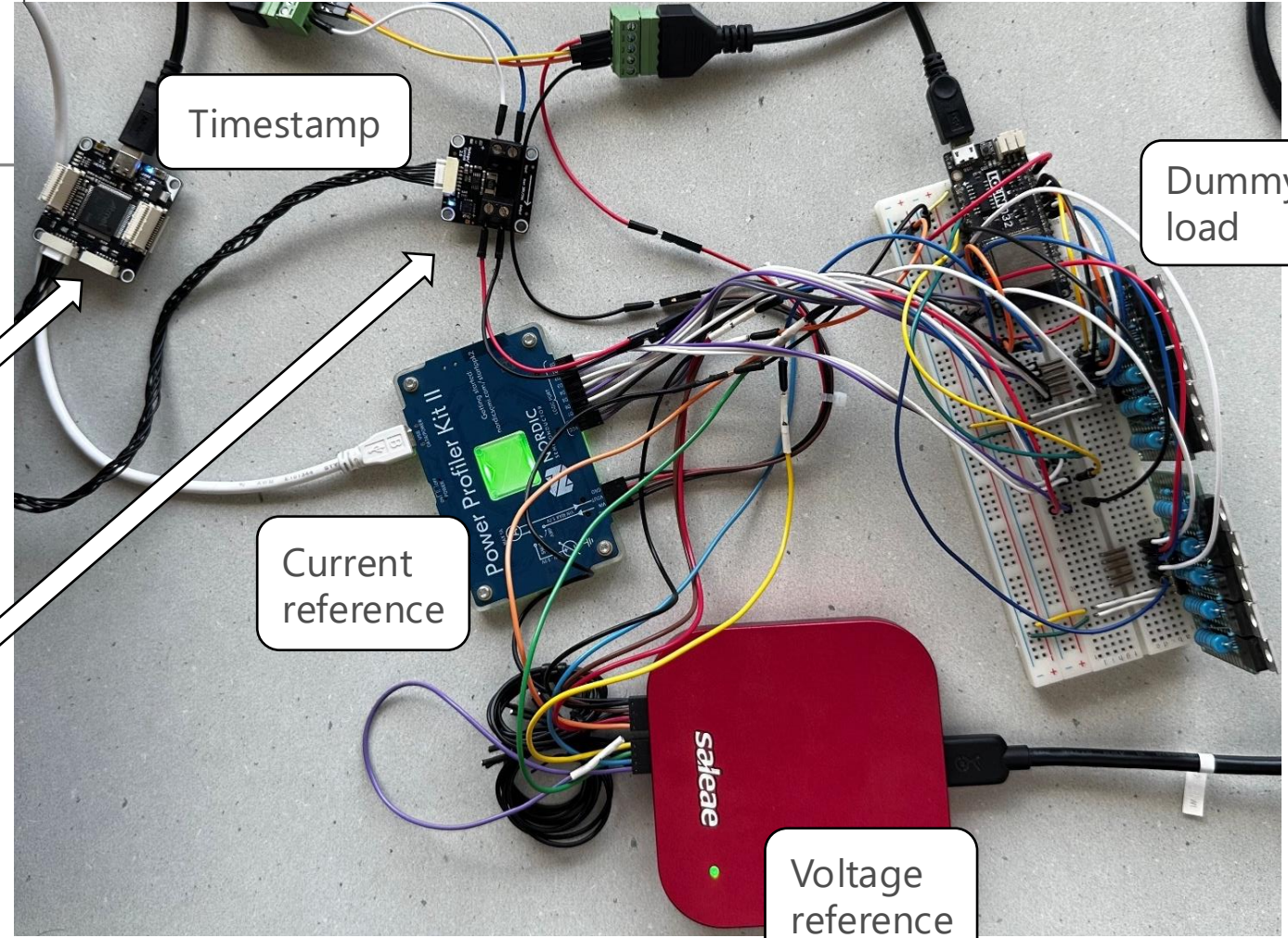


Master Brick

SPI



Voltage/Current Bricklet 2.0 Brick



Measurements of Reference Signal

- ▶ 100mA load resistors toggled on and off with increasing pulse duration starting with $P=5.0\text{ms}$
- ▶ Induce measurable voltage drop
- ▶ Accurate measurements for both current and voltage with our reference setup

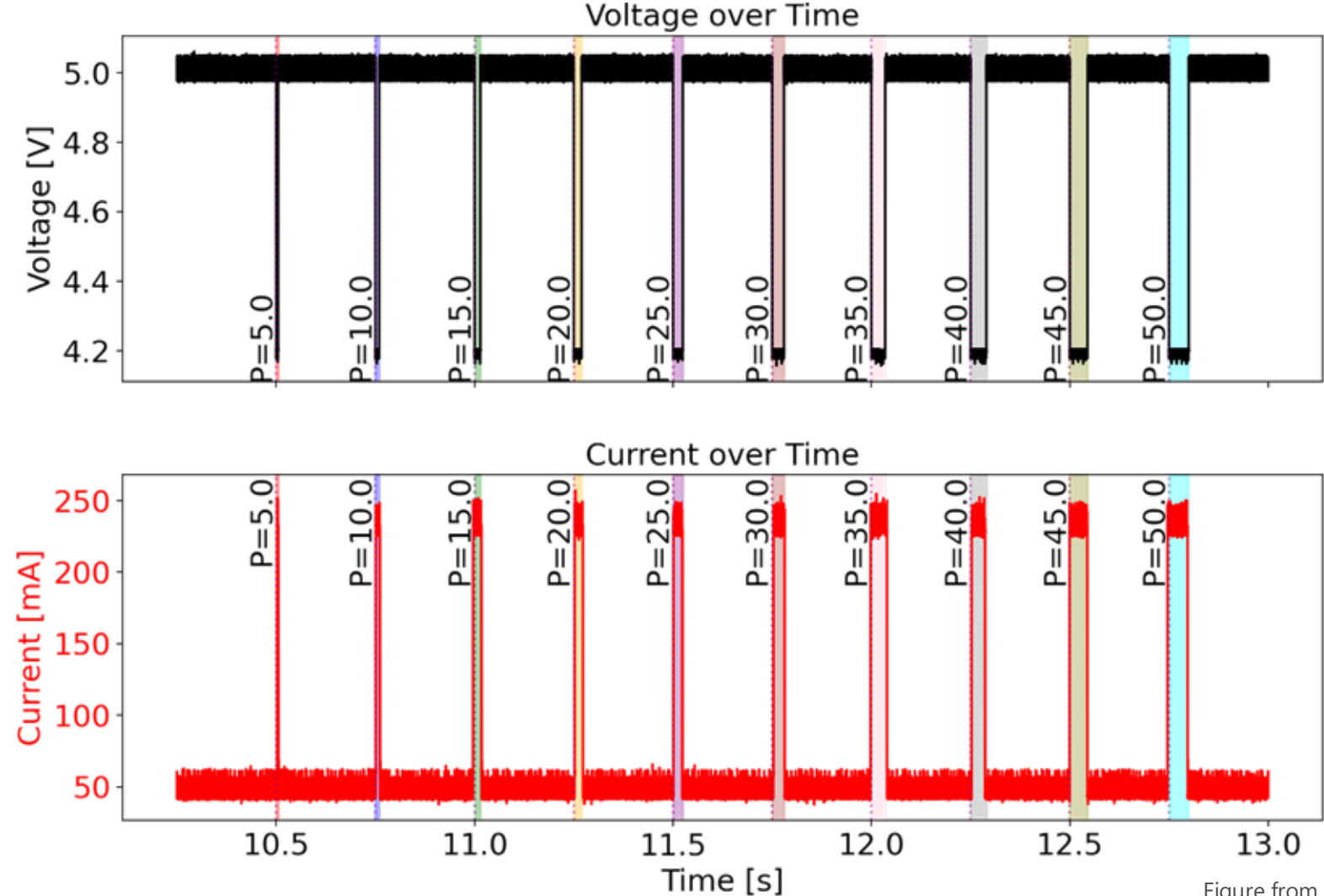
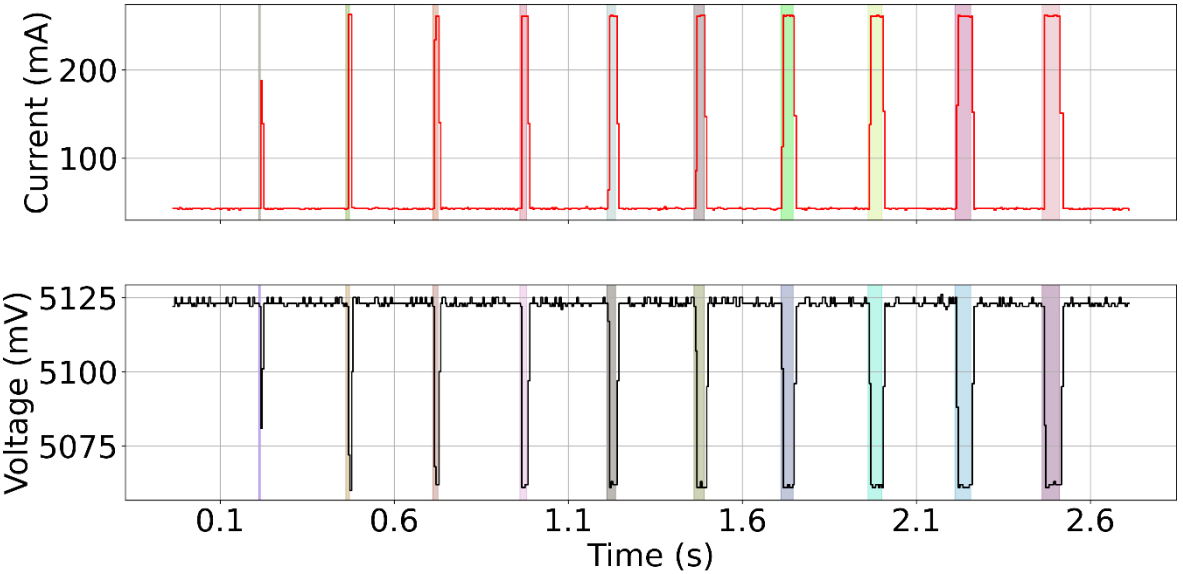


Figure from [1]

Comparison to Tinkerforge Setup

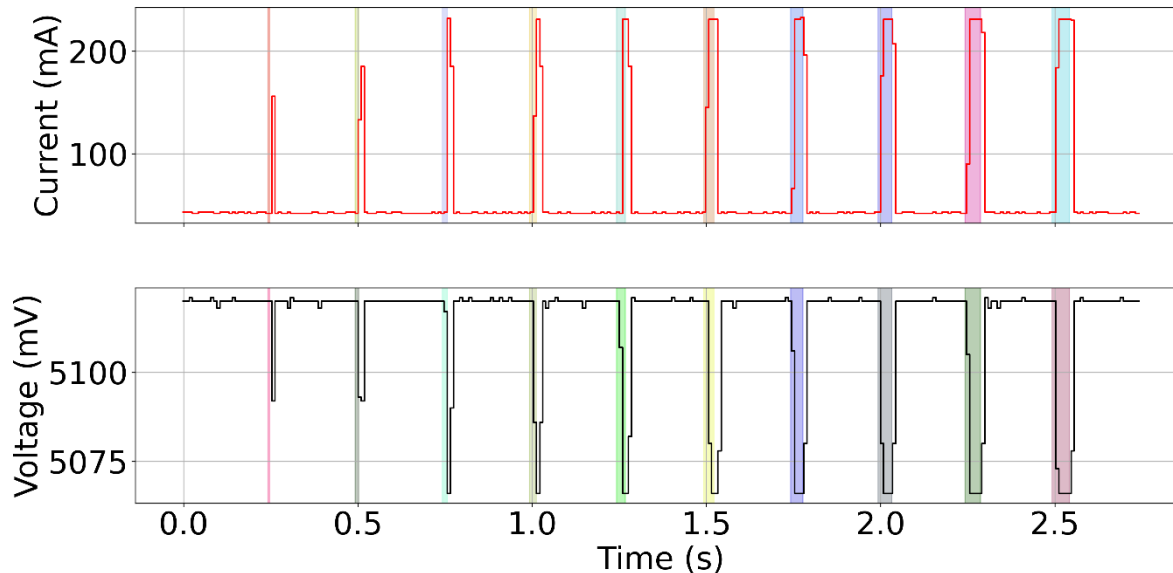
- ▶ Power spikes not accurately measured using the Tinkerforge setup
- ▶ **In addition:** timing inaccuracy possible
- ▶ Reference/validation measurements required e.g., double check using oscilloscope

Target time: 4.48ms

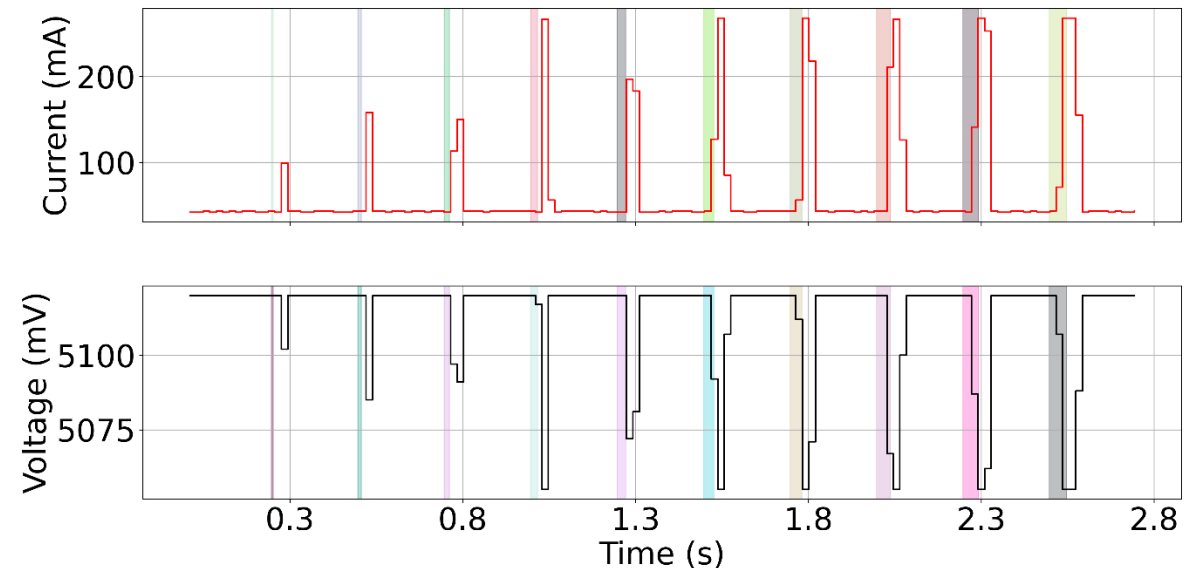


Figures from [1]

Target time: 8.8ms



Target time: 18.816ms



Impact for Real Devices and for Network Management Decisions

- ▶ Difference in measured energy consumption between Tinkerforge and reference setup
- ▶ **Difference in the range of -7.04% and 14.04%**
- ▶ **Average difference: 4.26%**
- ▶ **Assumption**
 - We run IoT devices transmitting 100B payload once an hour with different Radio Access Network technologies
 - We use the energy values from above and have the measured offset
 - We power the devices with typical coin cell batteries
- ▶ We would achieve runtime differences between +448 days and -729 days (nearly two years)

Target (ms)	Min. diff. (%)	Max. diff. (%)	Target (ms)	Min. diff. (%)	Max. diff. (%)
1.632	2.57	10.69	8.132	-5.43	10.54
2.200	-1.73	8.85	8.800	-2.87	11.08
2.656	0.32	14.04	10.336	-3.64	9.98
4.232	-0.78	8.81	16.488	-3.06	11.76
4.480	1.01	9.99	16.928	-2.74	11.59
4.704	-0.29	11.44	18.816	-7.04	5.83

Results from [1]

Battery lifetime analysis in days

Battery type	Tech.	Ideal	-7.04%	+4.26%	+14.04%
CR2032 Coin Cell	2G	136.8	147.2	131.2	120.0
	4G	326.6	351.3	313.3	286.4
	NB-IoT	77.3	83.1	74.1	67.8
	LTE-M	158.2	170.2	151.7	138.7
18650 Li-ion	2G	2481.1	2669.0	2379.7	2175.6
	4G	5922.6	6371.1	5680.6	5193.4
	NB-IoT	1401.5	1507.7	1344.3	1229.0
	LTE-M	2868.8	3086.0	2751.5	2515.6

INTERNAL ENERGY MEASUREMENTS

Interfaces for Internal Energy Measurements

- ▶ Several devices support **internal energy measurements**
- ▶ Specific hardware sensors and interfaces required
- ▶ **CPU, e.g., Running Average Power Limit (RAPL) interface**
 - Introduced with Intel's Sandy Bridge architecture
 - Also supported by modern AMD processors
 - Model-based estimates of energy consumption from hardware performance counters
 - Up to 1ms and micro-joules granularity
 - Can be accessed using Linux powercap framework or libraries like PyRAPL
- ▶ **GPU**
 - Instantaneous power measurements available: power_usage for NVIDIA, amd-smi for AMD
 - Both power (in Watts) and Energy (in Joules) values available
 - Typically, lower granularity: 20ms – 100ms and milli-joules
 - Total energy consumption achieved by
 - Sampling power values at regular intervals and integrated over time
 - Use summed up values: total_energy_consumption for NVIDIA, cumulative_energy for AMD

Selected Internal CPU Measurements

► PyRAPL [7]

- Python based approach to quantify energy from using Intel's Sandy Bridge architecture
- Simple and straight forward

► Scaphandre [8]

- Based on RAPL but more robust than PyRAPL
- Rust-based power consumption exporter

► CodeGreen [9]

- Modular, open-source energy measurement system
- Profile software energy consumption by decoupling instrumentation from measurement
- Using hardware sensors like Intel RAPL, NVIDIA NVML, or AMD ROCm

► Kepler [10]

- Measures energy consumption at the container, pod, VM, and process level.
- Uses RAPL to collect energy data from CPU packages, cores, memory subsystem
- Distributes energy proportionally to workloads based on CPU time consumption

```
pip install pyRAPL
```

```
import pyRAPL
```

```
pyRAPL.setup()
```

```
measurement = pyRAPL.Measurement("name")
```

```
measurement.begin()
```

```
# code
```

```
measurement.end()
```

```
result = measurement.result
```

Selected Internal GPU Measurements

► PyNVML [11]

- Access to NVIDIA Management Library (NVML) using Python
- Get instantaneous power and cumulative energy

```
pip install nvidia-ml-py
```

```
import pynvml
```

```
pynvml.nvmlInit()
```

```
handle = pynvml.nvmlDeviceGetHandleByIndex(0)
```

► PySMI [12]

- AMD GPUs (ROCm) equivalent to NVML
- Possible to get energy counter, average power consumption over short or long timeframe

```
energy = pynvml.nvmlDeviceGetPowerUsage(handle)
```

```
vram = pynvml.nvmlDeviceGetMemoryInfo(handle).used
```

► Jolt [13]

- Rust-based tool for micro-benchmarking GPU power spikes
- Clean GUI, also usable for larger systems or laptops
- Information about battery status, power draw from GPU (also from CPU), and process tracking

Selected Frameworks and Comprehensive Solutions

► CodeCarbon [14]

- **Goal:** track and reduce carbon emission from local computing
- Tracks energy for Intel/AMD CPUs and NVIDIA GPUs
- Estimates carbon emission based on local power grid

► Zeus [15]

- Optimized to improve deep learning solutions
- Measures energy demand and helps to find optimal energy configuration for GPU training, e.g., automatically adjusting power limits

► Eco2AI [16]

- Similar to CodeCarbon
- Focus is on high-precision tracking for AI models

► For all approaches: understand the methodology and frameworks

- Check requirements and support
- Check precision, like for the Tinkerforge approach!

Cross OS and Cross Chip Approach: CodeCarbon

- ▶ Example code with visualization available for hands-on (https://github.com/lsinfo3/noms_energy_tutorial)
- ▶ For this and other example: `pip install numpy pandas matplotlib psutil codecarbon`
- ▶ Energy monitoring test with CodeCarbon (`web_energy_monitor.py`)
 - **What does the example script**
 - 10s initial phase (not considered in final statistics)
 - 20s idle phase
 - 30s 'high CPU utilization' phase
 - **Execute the script and check your results**
 - Does the CPU utilization change?
 - Does the energy consumption change?
 - Why?

Visualization Results

- ▶ Many operating systems strictly guard access to physical hardware sensors, e.g., powermetrics
- ▶ **Without admin rights**
 - For security and privacy reasons, normal applications or user-level processes are completely blocked from reading the raw electrical current flowing through the CPU
 - Consequently, OS blocks CodeCarbon from seeing the real sensors using an IDE
 - CodeCarbon handles the block by entering estimation mode
 - It looks up, e.g., the Thermal Design Power (TDP) of your Apple M2 chip (in my case)
 - Simply outputs that constant, flat estimation (usually around ~48.5W) over and over again
 - It has no idea what the CPU is actually doing, so it draws a flat line.
 - Mapping function for Energy $E \approx f(x) = E[TDP]$
- ▶ **With admin rights**
 - Get info from powermetrics sensors directly
 - See the actual physical electricity being pulled from the battery by the processor and the constant estimation is turned off
 - Immediately see the dynamic wattage mirroring the real-time CPU utilization
 - Energy $E \approx \sum_{k=1}^N V[k] \cdot I[k] \cdot \Delta t + \epsilon$ with impact of ϵ to be determined

FROM INDIVIDUAL MEASUREMENTS TO SYSTEM ASSESSMENT

Device or Task Specific Energy Measurement

► Step 1: Select correct measurement setup

- Internal vs. external
- Select correct measurement device

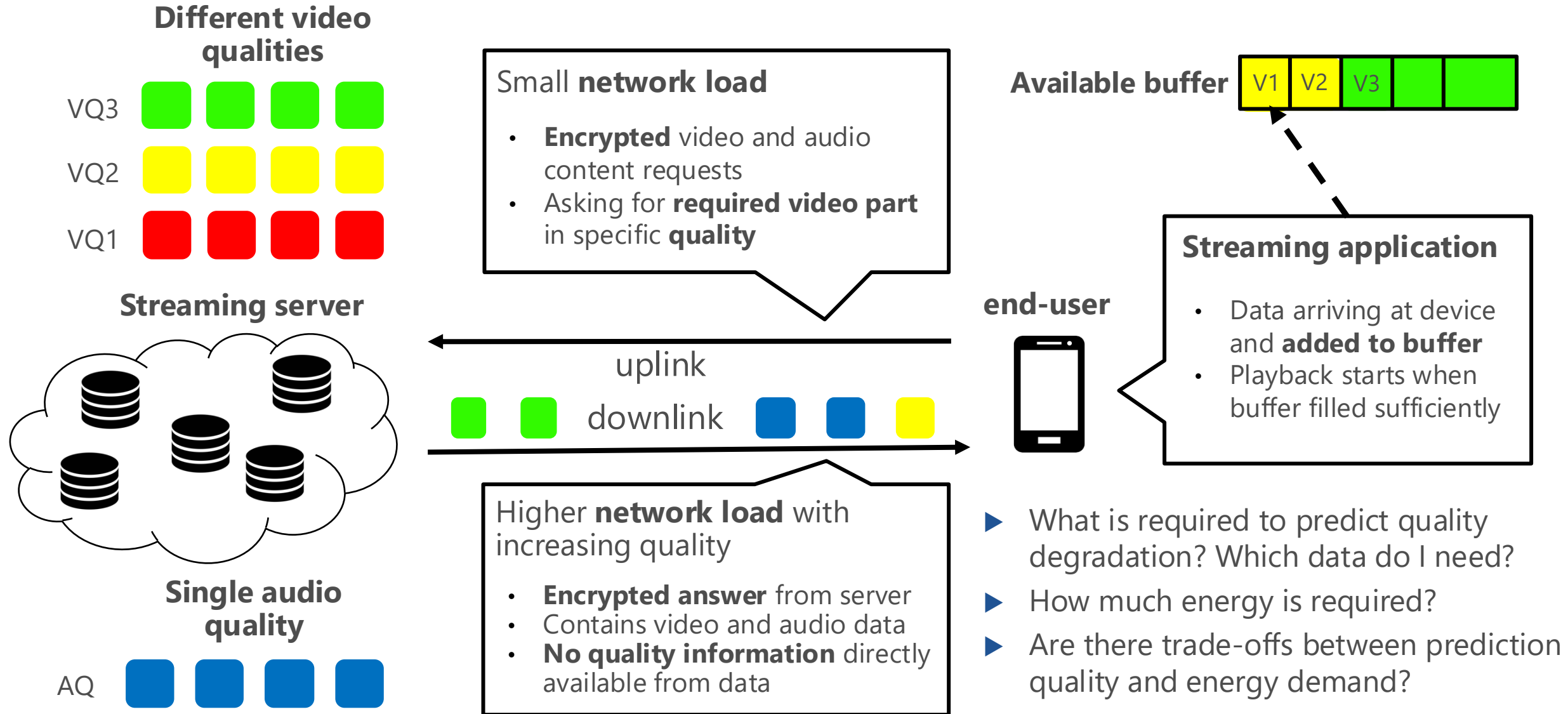
► Step 2: Isolate target process

- If possible: measure only target process
- If not possible
 - Limit background processes
 - Quantify energy demand of background processes

► Step 3: Comprehensive design of experiments

- **How long** do I need to measure, **which processes** do I want to measure?
- Differences between hot and cold runs – **any transient phase?** Batch means or replicate delete?
- **How many measurement runs** do we need to do, how large is my variance?
- If variance is very large – **is this correct?** Reasons for it?
- Are results statistically significant? Are they normal distributed? Can I use confidence intervals for comparison, or do I need other statistical tests?

Example: Video Streaming



Examples and results
from [17,18,19]

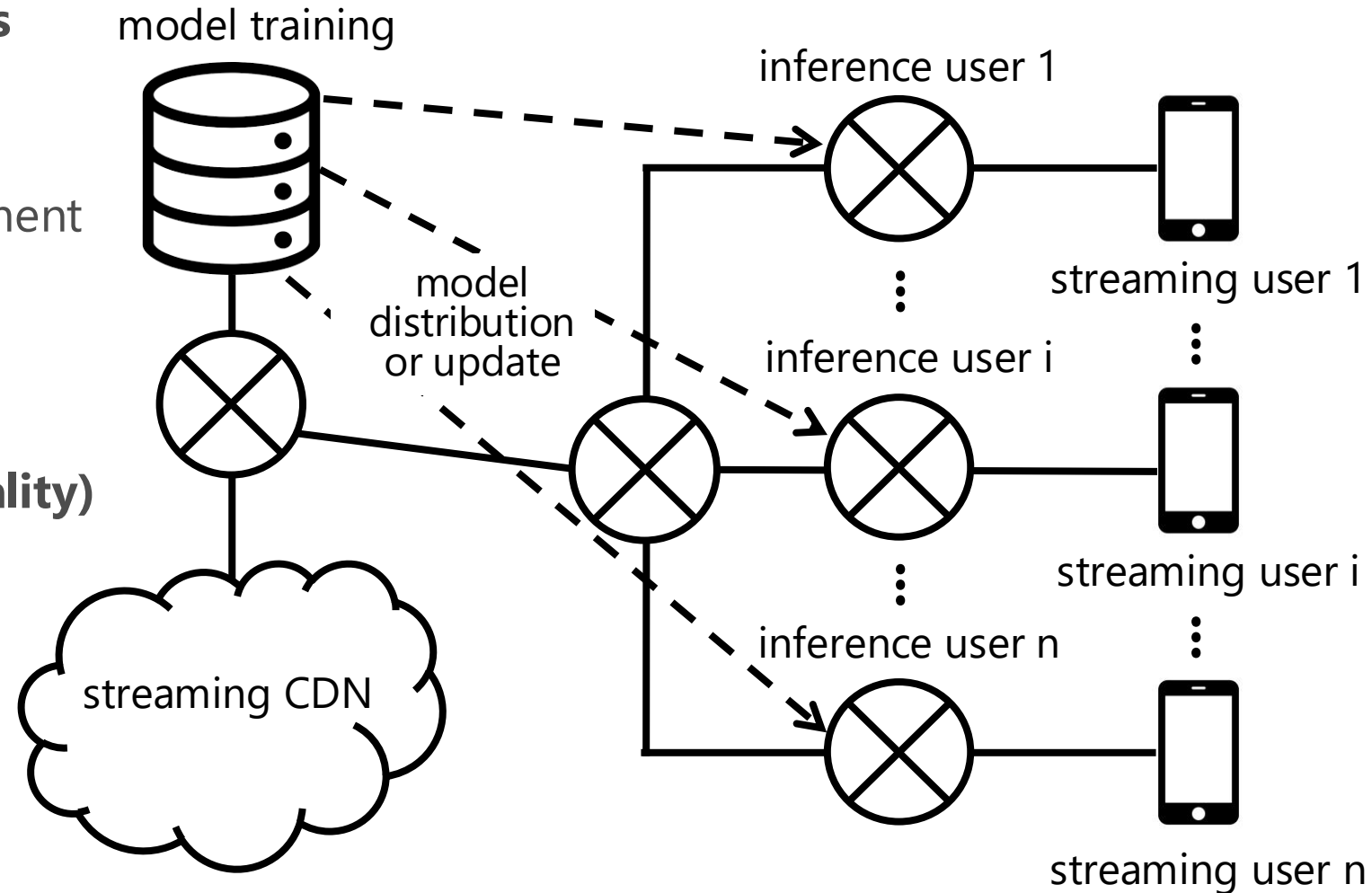
From Video Streaming to KQI Prediction

Streaming KQI prediction procedures

1. Capturing and processing of raw network traffic
2. Model selection and feature assessment
3. AI model training
4. Inference to predict streaming KQIs

Required to measure energy (and quality)

- ▶ Select correct hardware (GPU/CPU)
- ▶ Select correct energy measurement approach
- ▶ Select correct target KQI
- ▶ Select correct prediction approach
- ▶ Select input data (from [20])



Traffic Capturing and Processing

► Which packets do I need to capture and process?

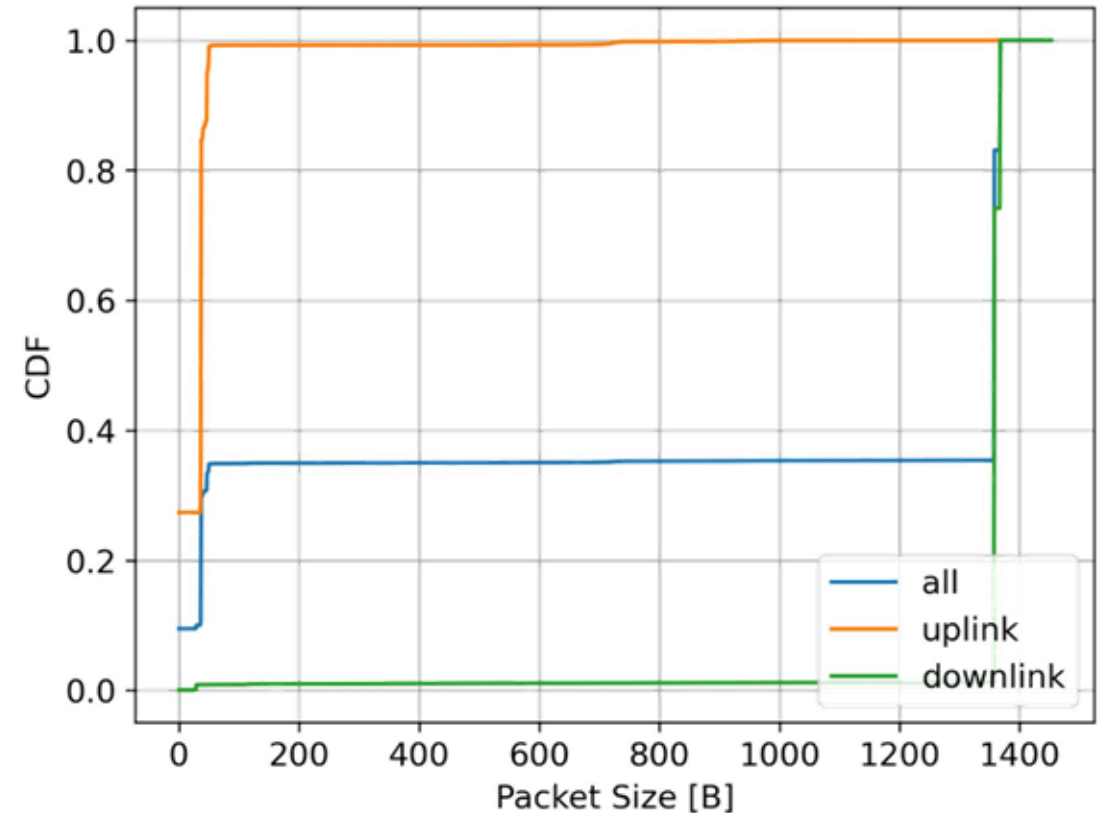
- Payload encrypted anyway?
→ **only headers**
- Also 'management' packets?
→ **only packets larger than x; what is x?**
- Traffic direction important?
→ **downlink, uplink or both?**

► Such decisions have massive impact on

- Required number packets
- CPU/RAM usage (hardware requirements)
- Energy requirements

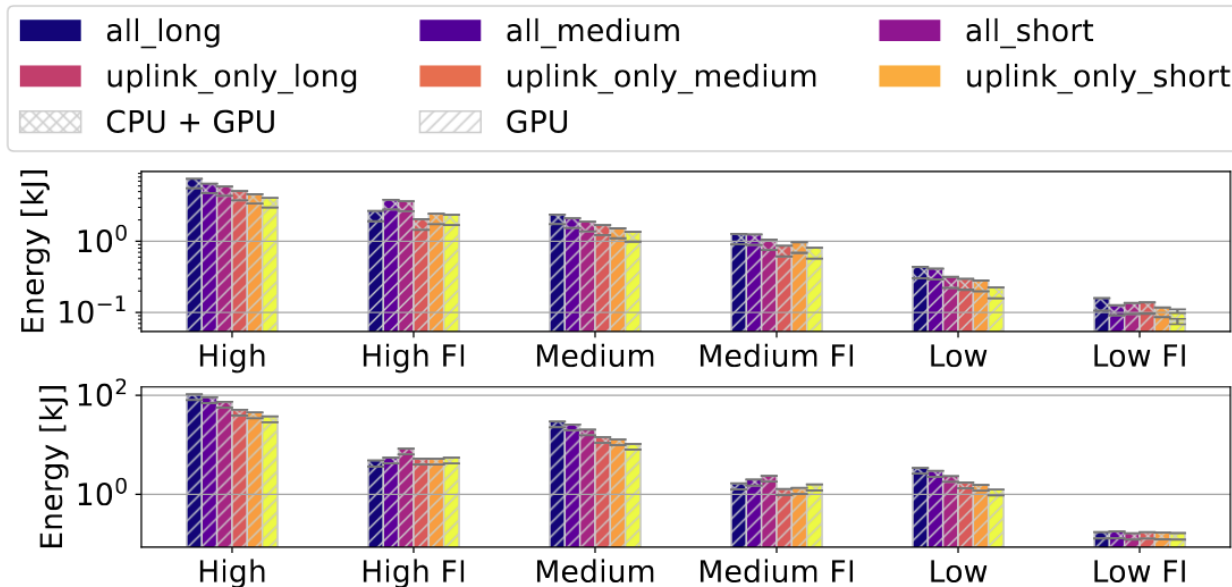
► Example assessments using PyRAPL (CPU) and PyNVML (GPU) for

- Training energy demand
- Inference energy demand and quality



Which Model, which Features

- ▶ **Deep learning or classical machine learning**
 - Model complexity: how many layers/trees etc.
 - Comprehensive hyperparameter optimization
- ▶ Which hardware: CPU or GPU
- ▶ Which and how many features, which KQI
- ▶ GPU also needs CPU support

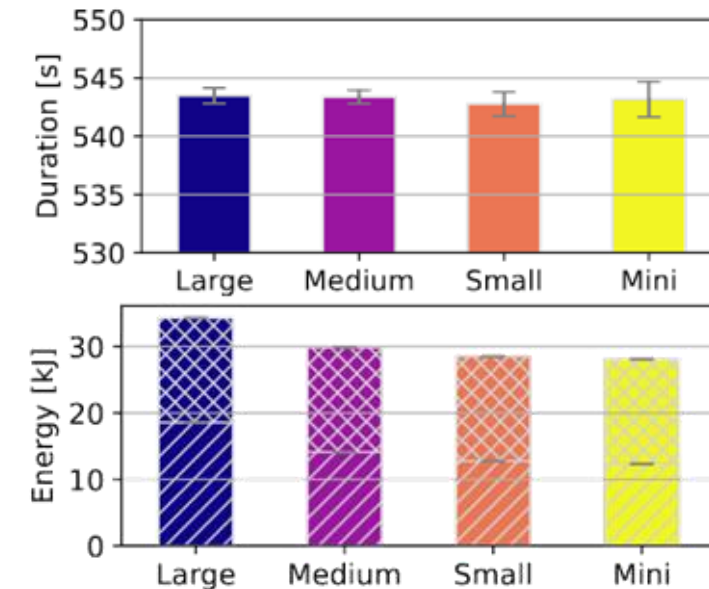


window lengths	up- and downlink		uplink only	
	name	No.	name	No.
1s, 2s, 3s, 5s, 10s, 20s	all_long	228	uplink_only_long	114
1s, 2s, 3s, 5s, 10s	all_medium	190	uplink_only_medium	95
1s, 2s, 3s	all_short	152	uplink_only_short	76

	all features	buffer health reduced	stalling reduced
all_long	228	10 (~ 4 %)	37 (~ 16 %)
all_medium	190	27 (~ 14 %)	39 (~ 21 %)
all_short	152	25 (~ 16 %)	37 (~ 24 %)
uplink_only_long	114	12 (~ 11 %)	40 (~ 35 %)
uplink_only_medium	95	24 (~ 25 %)	46 (~ 48 %)
uplink_only_short	76	33 (~ 43 %)	51 (~ 67 %)

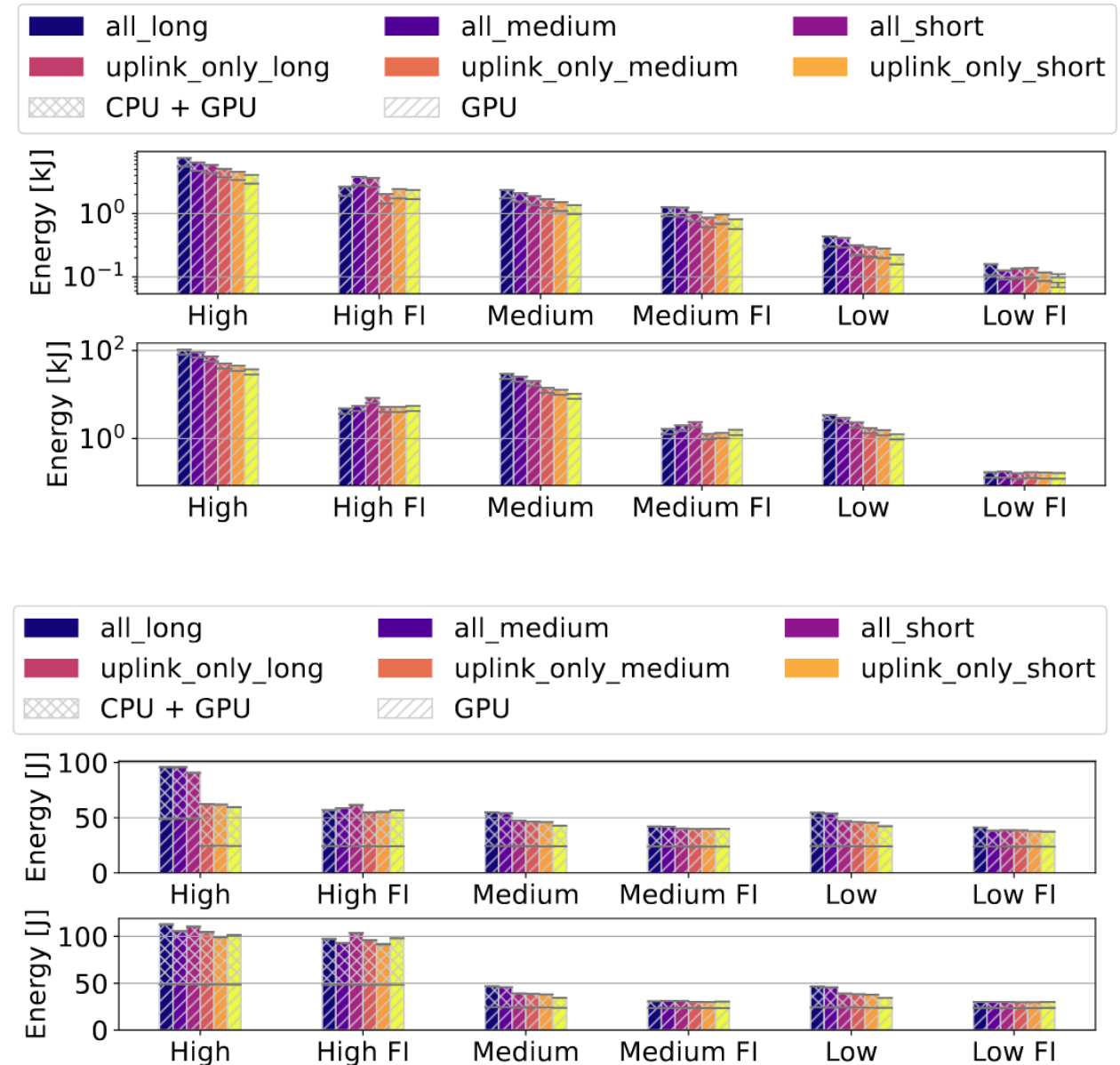
Model Training

Left, energy for **random forest** stalling classification (top) and buffer health regression (bottom), right **neural** stalling classifier



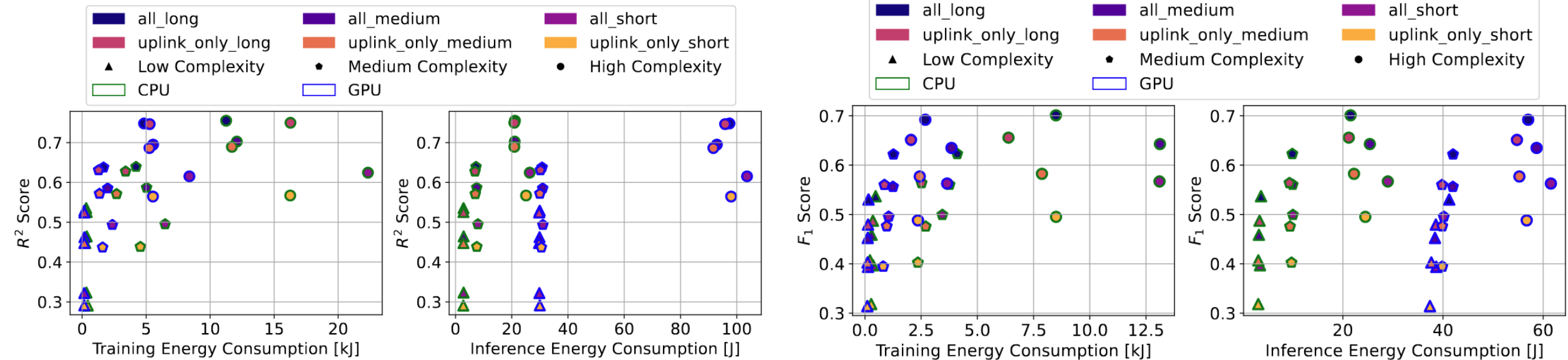
Training vs. Inference

- ▶ **Training typically consumes much more energy than inference!**
- ▶ But, performed
 - in single (small number of) datacenter(s)
 - significantly less often
- ▶ **Question:** use once more energy for training to save for inference in the long run?
 - Where are trade-offs?
 - How often re-training?
 - Best location for 'Green Model Training'?
- ▶ How much do I save during inference?
- ▶ What is my performance?



Example Performance for KQI Prediction in Streaming

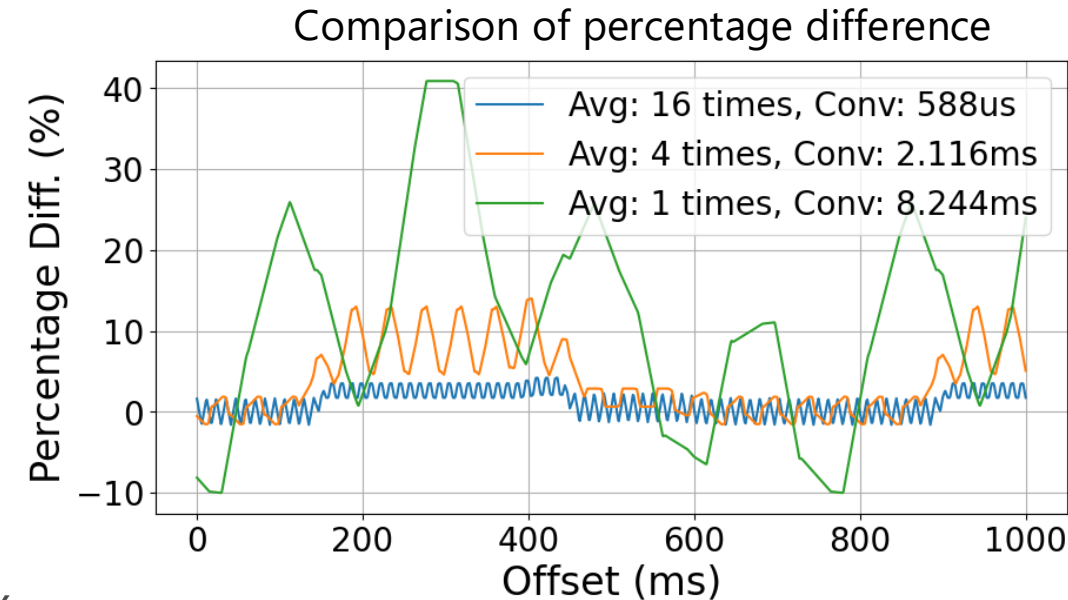
- Quality and energy demand for inference (buffer health regression left, stalling classification right)



- Lower complexity models tend to consume less energy
- Higher complexity models tend to perform better
- **However**
 - Specific low complexity models similar or cheaper than high complexity ones
 - GPU support can be beneficial – or require more energy for no improvement!

Conclusion on Energy Measurements

- ▶ **Precise energy measurements** essential for device and setup selection and network management
- ▶ Correct **usage and configuration** of setup important
- ▶ Increase number of measurement runs and duration to average out minor inaccuracies
- ▶ **Validation of measurement results is key!**
- ▶ **Usage of the correct measurement setup important**
 - Internal or external
 - Tinkerforge: low cost and simple with limitations
 - High precision lab equipment expensive and complex but achieves highly accurate results
 - Internal measurements not always supported. If supported, check precision!
- ▶ Trade-off between accuracy, complexity, and cost...
- ▶ ... but essential for high quality energy efficiency analysis.



Energy Measurements in Communication Networks: Discussion

► Where do I process my data, where do I train my models

- Distributed or centralized processing and training: in-network processing as solution?
- Does AI help with improving sustainability in communication networks or is 'the problem'?
- Large power plants for data centers to train AI models
 - Inference will require energy in our networks (distributed!)
 - More energy for training or inference
- Move the data or the energy – or both
 - Geographic movement
 - Time based movement

► More sustainable networks → with less energy demand?

- Network that consumes least energy is offline!
 - Trade-off among quality and energy requirements!
 - Network/system view vs. application view vs. user view → goal/target oriented
- Energy efficiency in communication networks

ENERGY EFFICIENCY IN COMMUNICATION NETWORKS

From Energy Consumption to Energy Efficiency

- ▶ How can we identify if a device, a system, or a network is energy efficient?



Ungefähr 5.620.000 Ergebnisse (0,05 Sek.)

- ▶ **'HowTo' energy efficiency in communication networks**
 - **Understand your system**
 - What does it do, what should it do?
 - What is the goal of the system?
 - Measure energy consumption and performance of the system
 - Select the correct energy efficiency metric

Energy Efficiency in Communication Networks

- ▶ Goal **from ICT perspective**: consider complete network, service operation, task, or application
- ▶ Two key 'cheats' assessing energy efficiency
 1. Assessment of **only parts of a network** required for a service

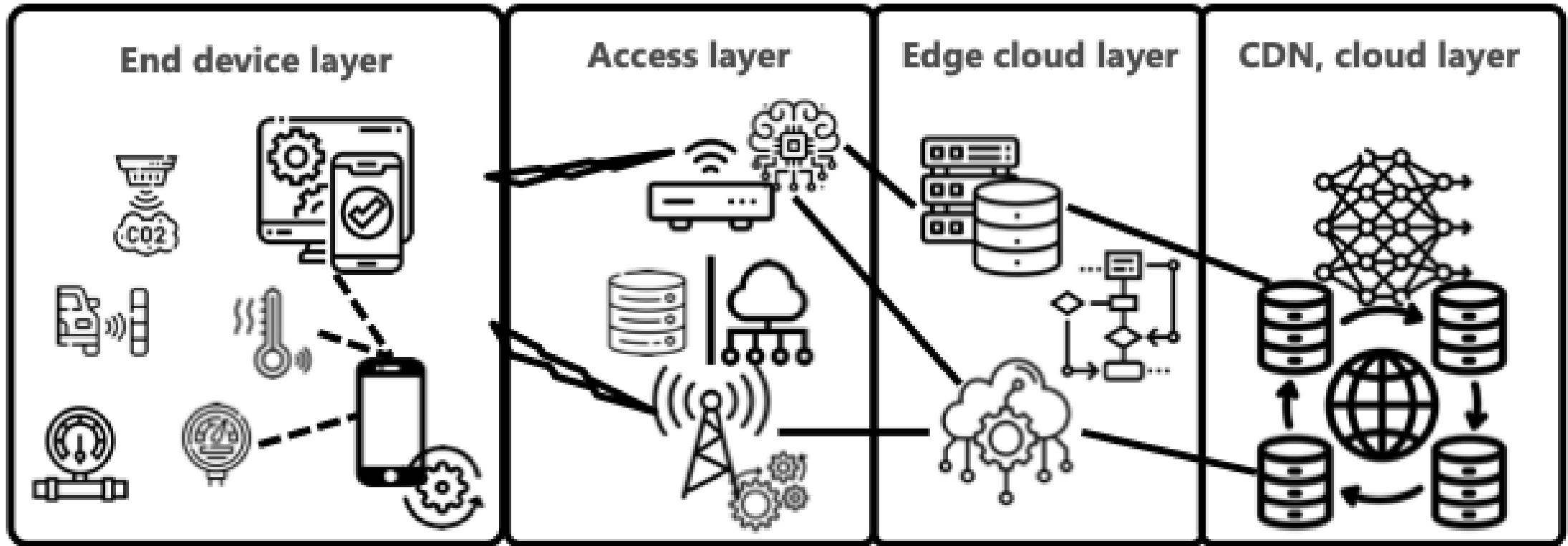


Figure from [4]

2. Usage of **the wrong (or best fitting)** energy efficiency metric

Some Examples from our Measurements

► Example use case consideration: data transmission on the Internet of Things

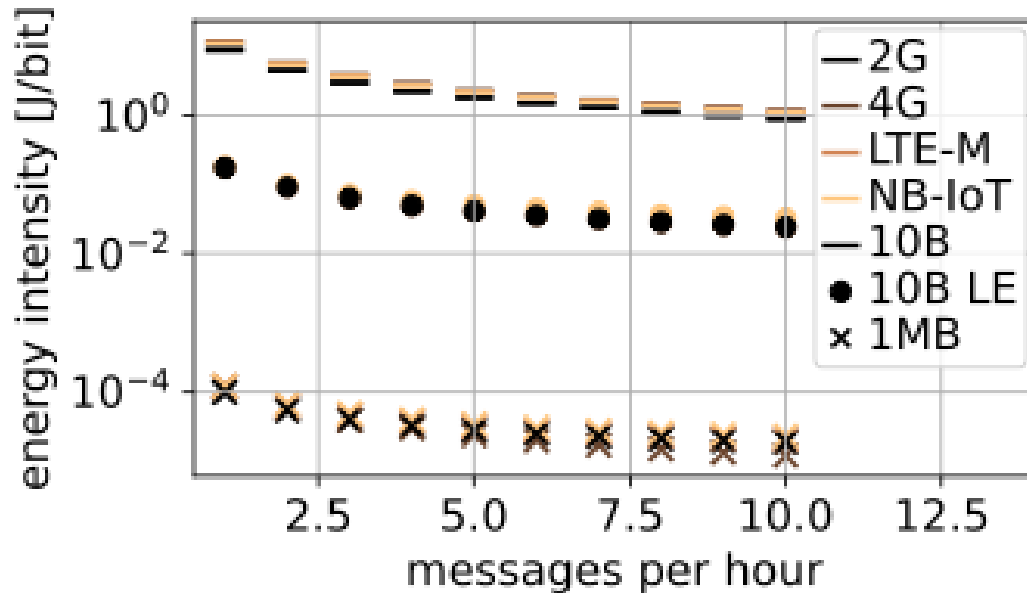
Payload	2G			4G			NB-IoT			LTE-M		
10 B	1.45 s	0.53 W	0.75 J	0.42 s	0.86 W	0.35 J	2.81 s	0.47 W	1.31 J	0.77 s	0.78 W	0.60 J
100 B	1.37 s	0.56 W	0.74 J	0.35 s	0.89 W	0.31 J	2.80 s	0.47 W	1.31 J	0.80 s	0.79 W	0.64 J
1 kB	1.80 s	0.53 W	0.92 J	0.37 s	0.81 W	0.30 J	4.36 s	0.47 W	2.01 J	1.25 s	0.80 W	0.90 J
10 kB	3.11 s	0.73 W	2.00 J	0.39 s	0.86 W	0.33 J	5.99 s	0.51 W	3.01 J	1.91 s	0.99 W	1.89 J
100 kB	8.09 s	1.25 W	9.63 J	0.73 s	1.06 W	0.77 J	23.93 s	0.66 W	15.64 J	9.19 s	1.16 W	10.67 J
1 MB	62.76 s	1.58 W	91.67 J	2.37 s	1.56 W	3.67 J	209.94 s	0.71 W	148.53 J	74.33 s	1.21 W	89.55 J

Results from [6]

- What is most energy efficient solution?
- Most energy efficient operation is **use case and goal dependent**
 - How many Bytes do I have to transmit?
 - Is my technology capable of transmitting that data?
- Implication for energy efficiency metric selection: do not forget
 - To consider idle, sleep, maintenance times
 - **Focus on target use case and best-case operation**

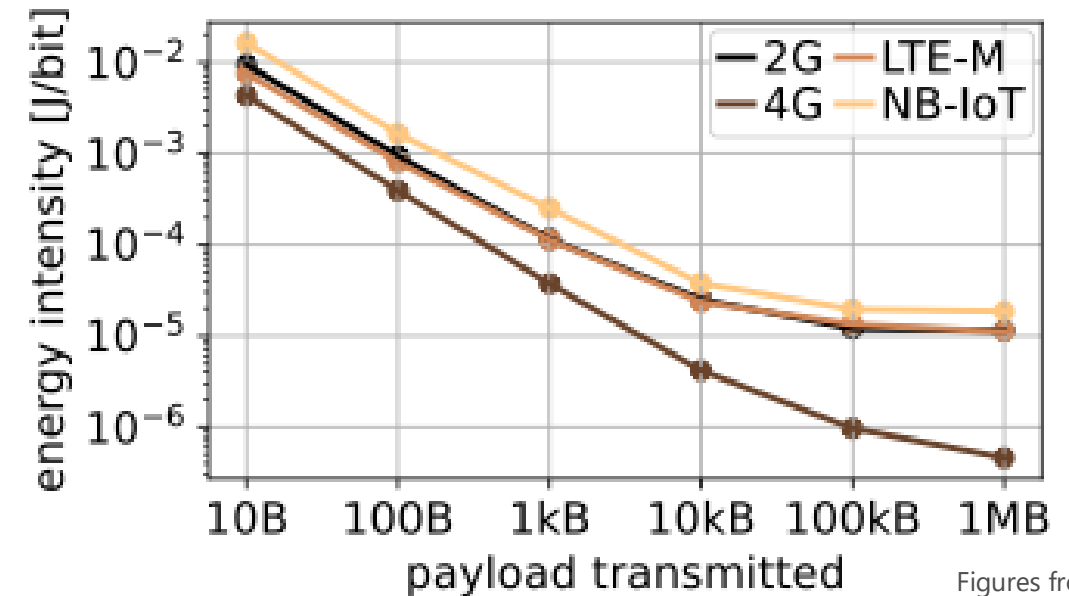
Simple and Popular Approach: Joule per bit (Energy Intensity)

- ▶ Variable number of transmissions per hour



- ▶ **Comparison among technologies** for same transmission rate and payload is valid
- ▶ More transmissions per hour **seem more energy efficient**
- ▶ Energy intensity **masks idle energy** with more transmissions

- ▶ Variable payload



Figures from [2]

- ▶ Larger payload seems more energy efficient
- ▶ Overhead per message is masked by payload
- ▶ Compare technologies **for same payload** or
- ▶ Only compare **same use case**, then assess total required energy!

Data Transmission in the Internet of Things: Example LoRaWAN

- Assessment of random access (RA), Time Scheduled (SC) and Listen before Talk (LBT)

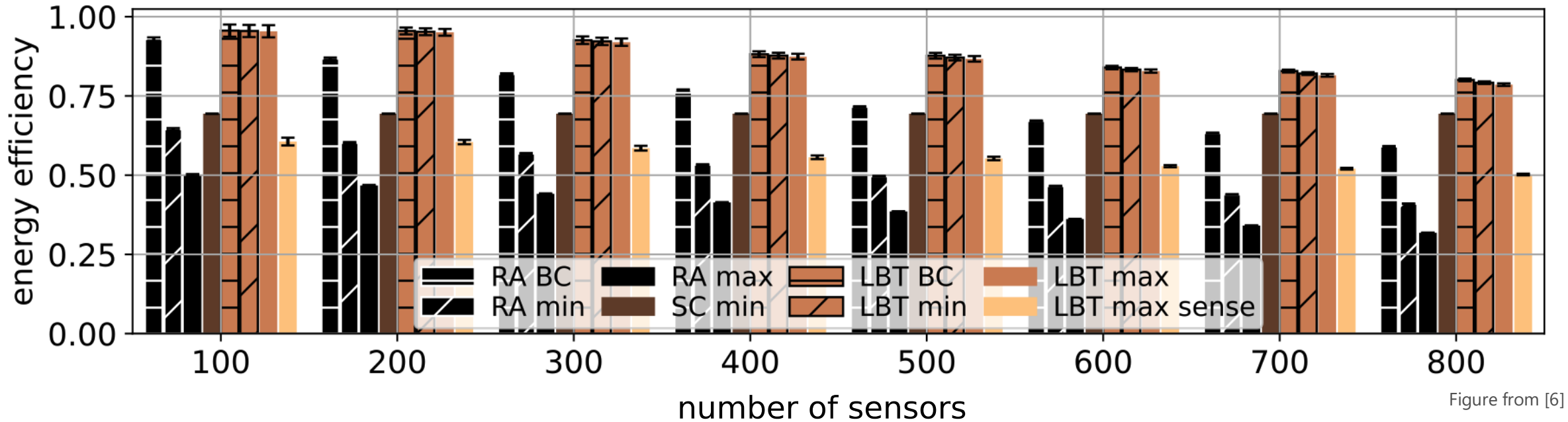


Figure from [6]

- **Ideally:** minimal energy for transmission, no collision, no synchronization required
- **Random access:** least expected energy demand not always most energy efficient solution!
- **Perfect scheduling:** no collisions thus, load in the network has no impact
- **Liste before Talk:** large energy consumption during listening on the channel
- **What is the goal?** – 300 sensors, collision probability less then 5%: **Random access best!**

HANDS-ON: ENERGY EFFICIENCY IN WIRELESS SENSOR NETWORKS

Hands-on: Energy Monitoring and Energy Efficiency Showcase

► Message transmission information

- Transmission from n independent sensors in a WSN in a time frame T using random access
- The collision probability p for random access within a total investigated time frame T can be achieved using $p = 1 - (\prod_{a \in S} (1 - \frac{T_s + T_a}{T}))$ considering transmissions s and a with transmission times T_s and T_a or simulated generating random transmission events and achieve overlapping transmission intervals
- The time to transmit a single 100B message is 0.02s and requires 10mW
- The time a sensor is idle requires 0.1mW

► Tasks: Calculate Energy Intensity (EI) as Joule per Byte

1. Calculate the average EI in the network if $n = [1000, 2000, 3000, 4000, \dots, 10000]$ and $T=3600s$, not considering message collisions or idle energy (one message per sensor per hour)
2. Only successfully arrived and no colliding messages are considered in EI calculation
3. Consider now also idle energy in your calculation
4. For $n=1000$, reduce T by 10%, 25%, 50%, 75%, 90%. This means, a sensor transmits more often per hour. How does energy intensity change.
5. What does this mean, e.g., for environmental outside temperature monitoring? Is the system really considered 'better' or 'more energy efficient' getting more values per time frame?

Code available: https://github.com/lsinfo3/noms_energy_tutorial

References

- [1]: F. Loh et al. (2026, May). **Assessment of Limitations in High Precision Energy Measurements for Low Power IoT Scenarios**, In Network Operations and Management Symposium. IEEE
- [2]: Loh, F., Amir, M., Mehling, N., Vomhoff, V., Blumenröder, J., & Hoßfeld, T. (2025, May). **Limitations Using Energy Intensity and General Energy Efficiency of 6G-Ready IoT Devices**. In Network Operations and Management Symposium (pp. 1-5). IEEE.
- [3]: Hossfeld, T., Wunderer, S., Loh, F., & Schien, D. (2024). **Analysis of Energy Intensity and Generic Energy Efficiency Metrics in Communication Networks: Limits, Practical Applications and Case Studies**. IEEE Access.
- [4]: Loh, F., Raffeck, S., Geißler, S., & Hoßfeld, T. (2023). **Paving the Way for an Energy Efficient and Sustainable Future Internet of Things**. Universität Würzburg.
- [5]: Hodkin, M. A., et al. (2024, June). **Energy-adaptive Network Switching via Intra-device Scaling**. In International Conference on Communications (pp. 2853-2858). IEEE.
- [6]: Loh, F., Raffeck, S., Geißler, S., & Hoßfeld, T. (2024). **Plan the access? Generic hardware independent energy consumption and efficiency model for different LoRaWAN channel access approaches**. IEEE Internet of Things Journal, 11, 19678-19693.
- [7]: <https://pypi.org/project/pyRAPL/>, Accessed: 2026-05-05
- [8]: <https://github.com/hubblo-org/scaphandre>, Accessed: 2026-05-05
- [9]: <https://codecarbon.io/>, Accessed: 2026-05-05
- [10]: <https://github.com/sustainable-computing-io/kepler>, Accessed: 2026-05-05
- [11]: <https://pythonhosted.org/nvidia-ml-py/>, Accessed: 2026-05-05
- [12]: <https://rocm.docs.amd.com/projects/amdsmi/en/latest/reference/amdsmi-py-api.html>, Accessed: 2026-05-05
- [13]: <https://github.com/jordond/jolt>, Accessed: 2026-05-05
- [14]: Rajput, S., & Sharma, T. (2026). **CodeGreen: Towards Improving Precision and Portability in Software Energy Measurement**. arXiv preprint arXiv:2603.17924.
- [15]: <https://ml.energy/zeus/measure/>, Accessed: 2026-05-05
- [16]: S. Budennyi et al. (2022, December). **Eco2ai: Carbon Emissions Tracking of Machine Learning Models as the First Step Towards Sustainable AI**. In Doklady mathematics (Vol. 106, No. Suppl 1, pp. S118-S128). Moscow: Pleiades Publishing.
- [17]: Loh, F., Baur, C., Raithel, F., Stüber, D., Ayoub, O., & Hoßfeld, T. (2025, October). **Energy Demand as AI Model Selection Criteria? Assessing Quality and Energy Consumption for AI based KQI Prediction in Video Streaming**. In 2025 21st International Conference on Network and Service Management (CNSM) (pp. 1-7). IEEE.
- [18]: Loh, F., Bingöl, G., Farahani, R., Pimpinella, A., Prodan, R., Atzori, L., & Hoßfeld, T. (2024, October). **High Complexity and Bad Quality? Efficiency Assessment for Video QoE Prediction Approaches**. In 2024 20th International Conference on Network and Service Management (CNSM) (pp. 1-5). IEEE.
- [19]: Loh et al. **GPU or CPU for Green AI? Performance and Energy Demand Assessment for AI-based Quality Prediction Approaches for Video Streaming**, under review.
- [20]: Loh, F., et al. (2022). **YouTube Dataset on Mobile Streaming for Internet Traffic Modeling and Streaming Analysis**. Scientific Data, 9(1), 293.